Lab 8 Setup and Run Guide: Deploying the Backend to ECS

lab08-ecs-backend.yaml builds the lab's infrastructure in one CloudFormation stack, so you can spend the lab time verifying and understanding it. The same work is described step by step in the lab README. The template follows the README's order, and its comments carry the README step numbers.

What the stack creates

README part	Resources
Part 1: reach AWS without a NAT Gateway	Endpoint security group (443 from the app security group only); interface endpoints for ecr.api, ecr.dkr and logs with private DNS; a private route table with the local route only, attached to both private subnets; an S3 gateway endpoint.
Part 2: stable entry point	Load balancer security group (80 from anywhere); target group (target-type ip, health check /actuator/health, matcher 200,401); Application Load Balancer in the public subnets; listener on port 80; a rule letting the load balancer into your app security group on the container port.
Part 3 and 4: managed service	ECS Fargate service in the private subnets, no public IP, attached to the target group, rolling deployment with minimum healthy 100% and maximum 200%.

It does not create the VPC, subnets, app security group, ECR image, cluster, or task definition. Those come from Modules 5, 6 and 7 and must already exist.

Before you start

- The image in ECR and the Module 7 cluster and task definition from module 7 setup.
- Your app security group from Module 7, and your team number (1 to 4).
- Use us-east-1 region.

Setup

Step 1. Open CloudShell and upload the template

Open CloudShell in your region. Use Actions, then Upload file, and choose lab08-ecs-backend.yaml.

Step 2. Collect the values the template needs

```
export REGION=us-east-1
export TEAM=1                  # 1, 2, 3 or 4

# Your VPC, and its subnets
export VPC=$(aws ec2 describe-vpcs --region $REGION --filters "Name=tag:Name,Values=leap-team$TEAM-vpc" --query "Vpcs[0].VpcId" --output text)

echo $VPC
```

```
# Your app security group from lab 07 (lab7-teamX-AppSecurityGroup)
aws ec2 describe-security-groups --region $REGION --filters Name=vpc-id,Values=$VPC \
 --query "SecurityGroups[].[GroupId,GroupName]" --output table

# The Module 7 task definition, by name
# arn:aws:ecs:us-east-1:760144414473:task-definition/leap-mission-service-teamX:X

aws ecs list-task-definitions --region $REGION --sort DESC \
--query "taskDefinitionArns[]" --output table
```

Public subnets are the ones with MapPublicIpOnLaunch true, and the rest are private. Note which two of each you will use. Task definition names and revisions vary, so use the ARN that the last command returns, including its revision number.

Step 3. Export the parameters

Export the following parameters:

```
export CNAME=mission-service
export CPORT=8080
Export SG=< the lab 7 security group>
export TASKDEF=<the taskDefinitionArn from step 2>


# Export your PUB and PRIV subnets

export PUB=$(aws ec2 describe-subnets --region $REGION --filters Name=vpc-id,Values=$VPC \
"Name=tag:Name,Values=leap-team$TEAM-public-*" --query "Subnets[].SubnetId" --output text | tr '\t' ',')

export PRIV=$(aws ec2 describe-subnets --region $REGION --filters Name=vpc-id,Values=$VPC \
"Name=tag:Name,Values=leap-team$TEAM-private-*" --query "Subnets[].SubnetId" --output text | tr '\t' ',')  echo $PUB;

echo $PRIV
```

Step 4. Deploy the stack

```
aws cloudformation deploy --region $REGION --template-file lab08-ecs-backend.yaml --stack-name lab8-
team$TEAM --tags team=$TEAM --parameter-overrides Team=$TEAM VpcId=$VPC PrivateSubnetIds=$PRIV
PublicSubnetIds=$PUB AppSecurityGroupId=$SG TaskDefinitionArn=$TASKDEF ContainerName=$CNAME Con-
tainerPort=$CPORT ClusterName=leap-mission-cluster-team$TEAM
```

This takes several minutes. The interface endpoints and load balancer take the longest. If your private subnets already have a route table with no internet route, add PrivateRouteTableId=rtb-xxxx so the stack uses it instead of creating a new one.

Verify, part by part

Step 5. Part 1: endpoints and routes

```
aws ec2 describe-vpc-endpoints --region $REGION \
 --filters Name=vpc-id,Values=$VPC Name=tag:aws:cloudformation:stack-name,Values=lab8-team$TEAM \
 --query "VpcEndpoints[].[ServiceName,VpcEndpointType,State]" --output table
```

Expect ecr.api, ecr.dkr and logs as Interface, s3 as Gateway, all available. Note there is still no route to the internet from the private subnets: the private route table should show only the local route and the S3 prefix list, and no 0.0.0.0/0.

Step 6. Part 2: load balancer and target group

```
export TG=$(aws cloudformation describe-stacks --region $REGION --stack-name lab8-team$TEAM \
 --query "Stacks[0].Outputs[?OutputKey=='TargetGroupArn'].OutputValue" --output text)
aws elbv2 describe-target-groups --region $REGION --target-group-arns $TG \
 --query "TargetGroups[0].[TargetType,HealthCheckPath,Matcher.HttpCode]"
```

Expect ip, /actuator/health and 200,401.

Step 7. Part 3: the service reaches a steady state

```
aws ecs describe-services --region $REGION --cluster leap-mission-cluster-team$TEAM \
 --services leap-mission-service \
 --query "services[0].[runningCount,desiredCount,deployments[0].rolloutState]"
aws elbv2 describe-target-health --region $REGION --target-group-arn $TG \
 --query "TargetHealthDescriptions[].[Target.Id,TargetHealth.State]" --output table
```

Expect 2, 2 and COMPLETED, and both targets healthy. It can take a couple of minutes after the stack finishes.

Step 8. Call the load balancer from your own machine

Run this from a terminal on your laptop (not CloudShell), using the load balancer's DNS name and not a task IP:

aws cloudformation describe-stacks --region $REGION --stack-name lab8-team$TEAM --query "Stacks[0].Outputs[?OutputKey=='AlbDnsName'].OutputValue" --output text

```
ALB=<AlbDnsName from the stack outputs>
curl -i http://$ALB/actuator/health
```

A 401 is the expected answer: the request got through the public load balancer to a task in a private subnet, and the application replied. A 502 or 503 means no healthy targets yet, and a timeout means the load balancer security group or listener is wrong.

Step 9. Part 4: deploy the service

```
aws ecs update-service --region $REGION --cluster leap-mission-cluster-team$TEAM \
 --service leap-mission-service --force-new-deployment
```

Troubleshooting

Symptom	Likely cause and fix
Tasks stop with "CannotPullContainerError"	The task cannot reach ECR. Check that all three interface endpoints and the S3 gateway endpoint are available, that the endpoint security group allows 443 from your app security group, and that the app security group allows outbound 443.
Stack create fails on the route table association	A private subnet already has its own route table. Rerun with PrivateRouteTableId set to that table's ID.
Endpoint creation fails about private DNS	DNS support and DNS hostnames must both be on for the VPC.
Targets show unhealthy	Check what the health path really returns with no credentials. 200 and 401 are accepted; a 404 or 500 is not. Also confirm the app security group allows the container port from the load balancer.
Service creation fails about the container name or port	ContainerName and ContainerPort must match the task definition exactly (step 2 shows them).
Tasks keep restarting	Read the stopped task's reason in the ECS console and the log group in CloudWatch. With the circuit breaker on, a failing deployment rolls back automatically.
Stack delete fails on the security group	Network interfaces from endpoints or tasks are still releasing. Wait a few minutes and delete the stack again.

Cleanup

The interface endpoints and the load balancer are billed by the hour, so delete them when you are finished.

```
aws cloudformation delete-stack --region $REGION --stack-name lab8-team$TEAM
```

The README tells you to keep the private route table and the S3 gateway endpoint for later modules, since they cost nothing. This stack puts them in the same stack as everything else, so deleting it removes them too. If you want to keep them, recreate them in the next module or ask your instructor for a version of the template that splits them out. The cluster, task definition, log group and ECR images are not part of this stack and stay in place.

Questions worth sitting with

- The health check accepts 401 only because the application protects /actuator/health. What would you change in the application so the load balancer could use a clean 200 instead?
- If a later extension had to call an external API from these private subnets, would PrivateLink be enough? What would you add, and would it replace the endpoints or sit beside them?